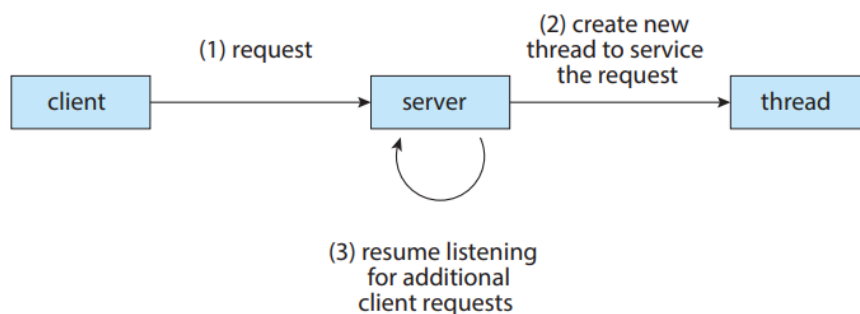


# **CHAPTER - 04**

**Question 1: Provide three programming examples in which multithreading provides better performance than a single-threaded solution.**



**Figure 4.2** Multithreaded server architecture.

## **1. Web Server:**

A web server can service each client request in a separate thread, allowing it to handle multiple requests simultaneously and improving overall performance compared to a single-threaded approach.

## **2. Parallelized Matrix Multiplication:**

Multithreading allows different parts of a matrix to be processed in parallel, significantly speeding up operations like matrix multiplication when compared to a single-threaded solution.

## **3. Interactive GUI Program:**

A debugger, for instance, can use one thread to monitor user input, another to run the application being debugged, and a third to monitor performance, ensuring the program remains responsive even with multiple tasks occurring at once.

**Question 2: Using Amdahl's Law, calculate the speedup gain of an application that has a 60 percent parallel component for (a) two processing cores and (b) four processing cores.\***

To calculate the speedup gain using Amdahl's Law, we use the formula:

$$\text{Speedup} = \frac{1}{S + \frac{(1-S)}{N}}$$

Where:

- $S$  is the serial portion of the application.
- $N$  is the number of processing cores.

The problem specifies a 60% parallel component, so the serial component  $S = 1 - 0.60 = 0.40$  (40%).

**(a) Speedup for two processing cores:**

For  $N = 2$ :

$$\text{Speedup} = \frac{1}{0.40 + \frac{(1-0.40)}{2}} = \frac{1}{0.40 + 0.30} = \frac{1}{0.70} \approx 1.43$$

**(b) Speedup for four processing cores:**

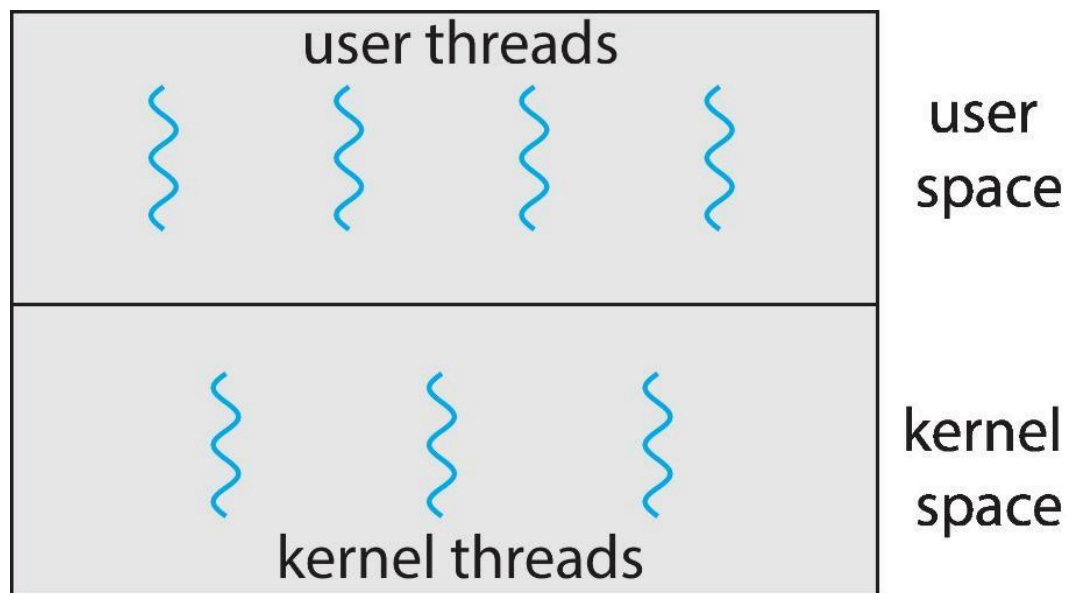
For  $N = 4$ :

$$\text{Speedup} = \frac{1}{0.40 + \frac{(1-0.40)}{4}} = \frac{1}{0.40 + 0.15} = \frac{1}{0.55} \approx 1.82$$

These results show the improvement in performance as the number of cores increases.

**Question 4: What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?\***

| Aspect      | User-Level Threads                                                                                                                                                                                                   | Kernel-Level Threads                                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Management  | Managed by user-level libraries, without OS involvement.                                                                                                                                                             | Managed and scheduled directly by the operating system (OS).                                                                                                                                                    |
| Performance | Faster to create and switch between threads, but cannot take advantage of multiple CPUs in parallel. <ul style="list-style-type: none"> <li>▪ Pthreads</li> <li>▪ Windows threads</li> <li>▪ Java threads</li> </ul> | Slower to create and manage, but can run on multiple processors in parallel. <ul style="list-style-type: none"> <li>• Windows</li> <li>• Linux</li> <li>• Mac OS X</li> <li>• iOS</li> <li>• Android</li> </ul> |



**User-level threads are better when:**

1. Fast thread creation and switching are important.
2. The application runs on a single-core processor without needing parallelism.
3. There is minimal interaction with the operating system or kernel services.

**Kernel-level threads are better when:**

1. The application requires true parallelism on multiple cores.
2. Frequent interaction with the operating system, like I/O operations, is necessary.
3. Robust concurrency control is required, managed directly by the OS.

### Question 5: Describe the actions taken by a kernel to context-switch between kernel level threads.

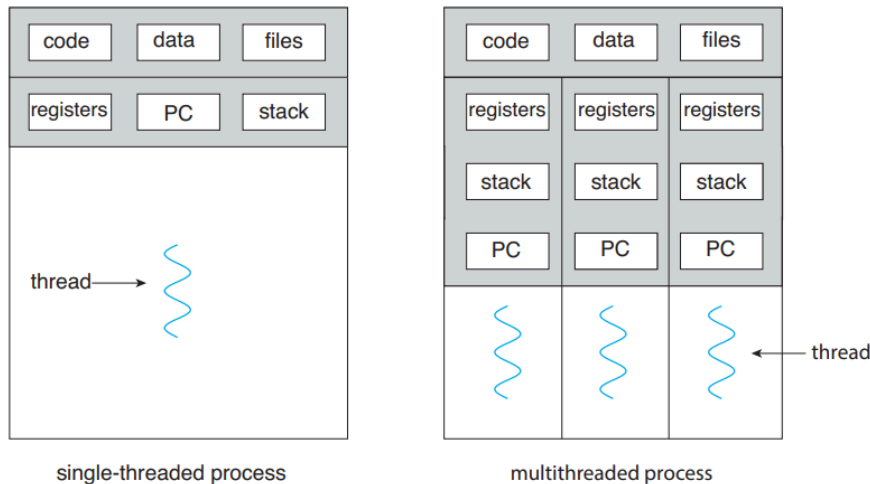
When a kernel performs a context switch between kernel-level threads, the following actions are typically taken:

1. **Save the state of the current thread:** The kernel saves the current thread's CPU register values, program counter, and other critical execution state information into the thread's control block.
2. **Update thread control block (TCB):** The thread's metadata, such as its state (e.g., ready, waiting, running), is updated to reflect that it has been switched out.
3. **Select a new thread to run:** The kernel's scheduler chooses the next thread to run based on scheduling algorithms (e.g., priority-based, round-robin).
4. **Restore the new thread's state:** The kernel loads the saved CPU register values, program counter, and other context information from the new thread's control block.
5. **Switch to the new thread:** The CPU begins executing the new thread's instructions, effectively completing the context switch.

### Question 6: What resources are used when a thread is created? How do they differ from those used when a process is created?

When a **thread** is created, the following resources are used:

1. **Thread Control Block (TCB):** A lightweight data structure that holds the thread's state information, such as:
  - ☐ Program counter (PC)
  - ☐ Stack pointer
  - ☐ CPU register set
  - ☐ Thread ID
2. **Stack:** Each thread has its own stack for managing function calls, local variables, and return addresses. The stack is crucial for controlling the execution flow of the thread.
3. **Registers:** A set of CPU registers is allocated to store the current execution state, such as the thread's program counter and instruction pointer.
4. **Thread-local Storage:** Threads can have their own storage for variables that are not shared with other threads but need to be kept local to the thread for its individual tasks.



**Figure 4.1** Single-threaded and multithreaded processes.

### When a process is created:

1. **Memory:**
  - o **Thread:** Shares memory (code, data, and heap) with other threads in the same process, but has its own stack.
  - o **Process:** Has its own separate memory space, including code, data, heap, and stack.
2. **Control Block:**
  - o **Thread:** Uses a lightweight **Thread Control Block (TCB)**.
  - o **Process:** Requires a larger **Process Control Block (PCB)**.
3. **Resource Sharing:**
  - o **Thread:** Shares resources (open files, environment variables) with the parent process.
  - o **Process:** Has its own set of independent resources.
4. **Context Switching:**
  - o **Thread:** Faster due to shared memory.
  - o **Process:** Slower as the entire memory space must be switched.

**Question 7: Assume that an operating system maps user-level threads to the kernel using the many-to-many model and that the mapping is done through LWPs. Furthermore, the system allows developers to create real-time threads for use in real-time systems. Is it necessary to bind a real-time thread to an LWP? Explain.**

Yes, it is necessary to bind a **real-time thread** to a **Lightweight Process (LWP)** in a **many-to-many model**, and here's why:

- In the **many-to-many model**, multiple user-level threads can be mapped to a smaller or equal number of kernel-level threads (LWPs). The operating system schedules these LWPs on actual processors.

- **Real-time threads** are used in **real-time systems**, where timing and responsiveness are critical. Real-time tasks must execute with minimal delay to meet strict deadlines.
- If a **real-time thread** is not bound to an LWP, it may have to wait to be assigned to one before it can run, introducing **unpredictable delays**. This could lead to missed deadlines, which is unacceptable in real-time systems.

By binding a real-time thread to an LWP, you ensure that the thread can run immediately when needed, without waiting for an available LWP, making the system more predictable and suitable for real-time operations.

### **Question 8: Provide two programming examples in which multithreading does not provide better performance than a single-threaded solution.**

#### **1. Sequential Programs:**

In tasks that must be completed in a strict sequence, such as calculating an individual tax return, multithreading adds unnecessary complexity without improving speed. Each step depends on the previous step, so threads cannot run concurrently.

#### **2. Shell Programs:**

Command-line shell programs don't benefit from multithreading because they need to closely manage their environment, such as open files and directories, making thread synchronization difficult and inefficient.

### **Question 9: Under what circumstances does a multithreaded solution using multiple kernel threads provide better performance than a single-threaded solution on a single-processor system?**

A multithreaded solution using multiple kernel threads provides better performance than a single-threaded solution on a single-processor system under the following circumstances:

1. **I/O-bound tasks:** If the program frequently performs I/O operations (like reading from a disk or network), a multithreaded solution allows one thread to perform CPU work while another is waiting for I/O. This reduces idle time and makes better use of the processor.
2. **Frequent blocking:** If a program experiences blocking events (such as waiting for resources or input), other threads can continue executing while one thread is blocked, improving overall performance by keeping the CPU busy.

In contrast, a single-threaded solution would stall during I/O or blocking, leading to inefficient use of CPU time.

**Question 10: Which of the following components of the program state are shared across threads in a multithreaded process?**

**(a) Register values (b) Heap memory (c) Global variables (d) Stack memory.**

In a multithreaded process, the following components are **shared** across threads:

**(b) Heap memory (c) Global variables**

The following components are **not shared** and are unique to each thread:

**(a) Register values (d) Stack memory**

**Question 11: Can a multithreaded solution using multiple user-level threads achieve better performance on a multiprocessor system than on a single-processor system? Explain.**

No, a multithreaded solution using **multiple user-level threads** will not achieve significantly better performance on a **multiprocessor system** compared to a **single-processor system**. Here's why:

- **User-level threads** are managed by the user's threading library, not the operating system. The OS is unaware of these individual threads and treats the entire process as one unit.
- On a **multiprocessor system**, the OS can only assign the whole process to **one core**, meaning all the user-level threads will still run on a single core, even if multiple cores are available.

Since user-level threads can't run on different cores simultaneously, there won't be a significant performance improvement on a multiprocessor system compared to a single-processor system. To utilize multiple cores, **kernel-level threads** (which are managed by the OS) would be needed.

**Question 12: In Chapter 3, we discussed Google's Chrome browser and its practice of opening each new tab in a separate process. Would the same benefits have been achieved if, instead, Chrome had been designed to open each new tab in a separate thread? Explain.**

No, the same benefits would not have been achieved if Chrome had opened each new tab in a separate thread instead of a separate process.

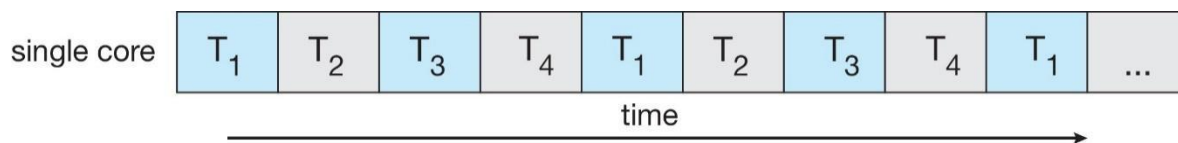
- **Separate Processes:** Each tab in Chrome is opened in a separate process, meaning if one tab crashes or experiences issues, it doesn't affect the other tabs or the entire browser. This enhances **stability** and **security**.

- **Separate Threads:** If each tab were opened in a separate thread within the same process, they would share the same memory space. This means if one tab crashes, it could potentially bring down the entire browser or affect other tabs.

By using separate processes for each tab, Chrome achieves better **isolation**, **stability**, and **security** than it would with separate threads.

### Question 13: Is it possible to have concurrency but not parallelism? Explain.

**Concurrency** refers to the concept of multiple tasks making progress at the same time. In a concurrent system, tasks can start, run, and complete in overlapping time periods. However, this doesn't mean the tasks are running simultaneously. Concurrency can be achieved on a single-core system by quickly switching between tasks. **Parallelism**, on the other hand, means that multiple tasks are being executed simultaneously on different processors or cores.



On a single-core system, concurrency is achieved by switching between tasks (T1, T2, T3, T4). The CPU executes part of a task, pauses it, and switches to another. However, only one task runs at a time, there is no parallelism.

### Question 14: Using Amdahl's Law, calculate the speedup gain for the following applications:

- 40 percent parallel with (a) eight processing cores and (b) sixteen processing cores

**1. 40% parallel ( $P = 0.40$ )**

(a) With 8 processing cores ( $N = 8$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.40) + \frac{0.40}{8}} = \frac{1}{0.60 + 0.05} = \frac{1}{0.65} \approx 1.54$$

(b) With 16 processing cores ( $N = 16$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.40) + \frac{0.40}{16}} = \frac{1}{0.60 + 0.025} = \frac{1}{0.625} \approx 1.60$$



- 67 percent parallel with (a) two processing cores and (b) four processing cores

### 2. 67% parallel (P = 0.67)

(a) With 2 processing cores ( $N = 2$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.67) + \frac{0.67}{2}} = \frac{1}{0.33 + 0.335} = \frac{1}{0.665} \approx 1.50$$

(b) With 4 processing cores ( $N = 4$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.67) + \frac{0.67}{4}} = \frac{1}{0.33 + 0.1675} = \frac{1}{0.4975} \approx 2.01$$

- 90 percent parallel with (a) four processing cores and (b) eight processing cores.

### 3. 90% parallel (P = 0.90)

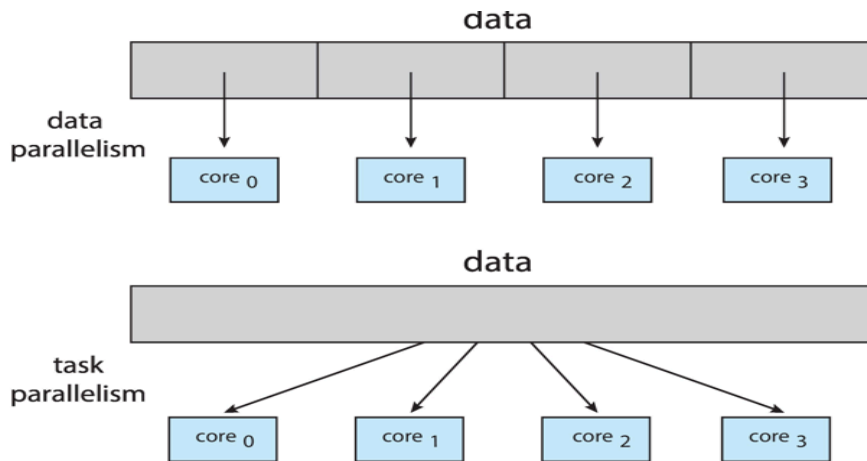
(a) With 4 processing cores ( $N = 4$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.90) + \frac{0.90}{4}} = \frac{1}{0.10 + 0.225} = \frac{1}{0.325} \approx 3.08$$

(b) With 8 processing cores ( $N = 8$ ):

$$\text{Speedup} = \frac{1}{(1 - 0.90) + \frac{0.90}{8}} = \frac{1}{0.10 + 0.1125} = \frac{1}{0.2125} \approx 4.71$$

**Question 15:** Determine if the following problems exhibit task or data parallelism:



- **Using a separate thread to generate a thumbnail for each photo in a collection**

**Data parallelism:** The same task (generating a thumbnail) is being applied to different pieces of data (photos).

- **Transposing a matrix in parallel**

**Data parallelism:** The same operation (transposing) is being applied to different parts of the matrix concurrently.

- **A networked application where one thread reads from the network and another writes to the network**

**Task parallelism:** Different tasks (reading and writing) are being performed by separate threads.

- **The fork-join array summation application described in Section 4.5.2**

**Data parallelism:** The summation operation is divided across different parts of the array, and each part is handled by separate threads.

- **The Grand Central Dispatch system**

**Task parallelism:** Different tasks are assigned to different threads and dispatched for parallel execution.

**Question 16:** A system with two dual-core processors has four processors available for scheduling. A CPU-intensive application is running on this system. All input is performed at program start-up when a single file must be opened. Similarly, all output is performed just before the program terminates,

when the program results must be written to a single file. Between start-up and termination, the program is entirely CPU-bound. Your task is to improve the performance of this application by multithreading it. The application runs on a system that uses the one-to-one threading model (each user thread maps to a kernel thread). \* (5)

i. **How many threads will you create to perform the input and output? Explain.**

- ☐ You only need **1 thread for input** and **1 thread for output**.
- ☐ Since the input happens once at the start and the output happens once at the end, there's no need for multiple threads. The I/O operations are straightforward and don't happen frequently during the main execution.

ii. **How many threads will you create for the CPU-intensive portion of the application? Explain.**

- ☐ You would create **4 threads** for the CPU-bound portion.
- ☐ The system has 4 cores available, so creating 4 threads ensures that each core is fully utilized. Each thread will be assigned to a core, allowing the program to run as fast as possible on the available hardware.

**Question 20: Consider a multicore system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be greater than the number of processing cores in the system. Discuss the performance implications of the following scenarios:**

- **The number of kernel threads allocated to the program is less than the number of processing cores.**

**Kernel Threads Less than Cores:**

- ☐ **Problem:** Not all cores are being used because there aren't enough kernel threads to fully use all the cores.
- ☐ **Effect:** Some cores will sit idle, and the program won't run as fast as it could.

- **The number of kernel threads allocated to the program is equal to the number of processing cores.**

#### Kernel Threads Equal to Cores:

- ☐ **Ideal Situation:** Each core has one kernel thread, so all cores are fully used.
  - ☐ **Effect:** This setup gives the best performance because all cores are working efficiently, though some overhead might happen when handling extra user threads.
- **The number of kernel threads allocated to the program is greater than the number of processing cores but less than the number of user-level threads.**

#### More Kernel Threads than Cores:

- ☐ **Issue:** The system has more kernel threads than cores, so the CPU will need to switch between them.
- ☐ **Effect:** This causes some performance loss due to **context switching**, but more tasks can be handled at once than in the first case. Performance won't be as good as when threads match cores, but it's better than having fewer threads than cores.